

Introducere în Godot

(Convenție - scrisul portocaliu are hyperlink-uri embedded către documentație, dacă dați click/ CTRL+click o să vă redirecționeze către documentația aferentă)

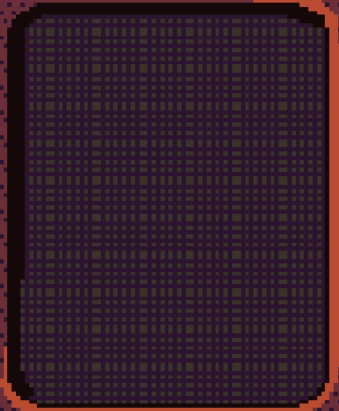
Prof. Coordonator: Ș.l.dr.ing.Stelian-Nicolae NICOLA

Mentor: Pătrașcu Hary

Ce este Godot ?

Un game engine open-source, cross-platform cu capabilități 2D și 3D dezvoltat în C++.

Godot permite folosirea a mai multor limbaje de programare (principal GDScript și C#) și publicarea jocurilor pe toate sistemele de operare moderne: Windows, Linux, MacOS, Android, iOS, dar și pe console.



GScript



GScript este un limbaj de programare high-level, orientat pe obiecte cu sintaxa similară limbajului Python.

Exemple de cod în GDScript

```
Debug      player_controller.gd      Online Docs      Search Help      < >

1  class_name PlayerController extends CharacterBody2D
2
3
4  @onready var tools_animated_sprite: AnimatedSprite2D = $ToolsAnimatedSprite
5  @onready var character_animated_sprite: AnimatedSprite2D = $CharacterAnimatedSprite
6
7  @export_category("speed variables")
8  @export var _speed: float = 100.0
9  @export var _sprint_speed: float = 150.0
10 var _current_speed: float = _speed
11
12 @export_group("input variables")
13 var _input_dir: Vector2 = Vector2.ZERO
14 var _can_move: bool = true
15
16 ▾ var _facing_right: bool = false:
17   ▾ > set(value):
18     > > _facing_right = value
19     > > character_animated_sprite.flip_h = not _facing_right
20     > > tools_animated_sprite.flip_h = not _facing_right
21
22 > enum Tool {
23   >
24 }
25
26 @export var _current_tool: Tool
27 @export var _tool_anims: Dictionary[Tool, String]
28
29
30 ▾ enum State {
31   > IDLE,
32   > WALKING,
33   > RUNNING,
34   > USING_TOOL
35 }
36
```

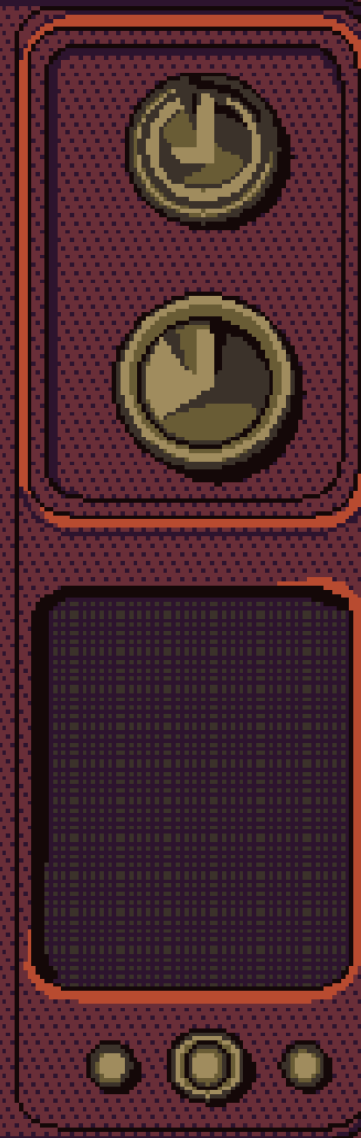
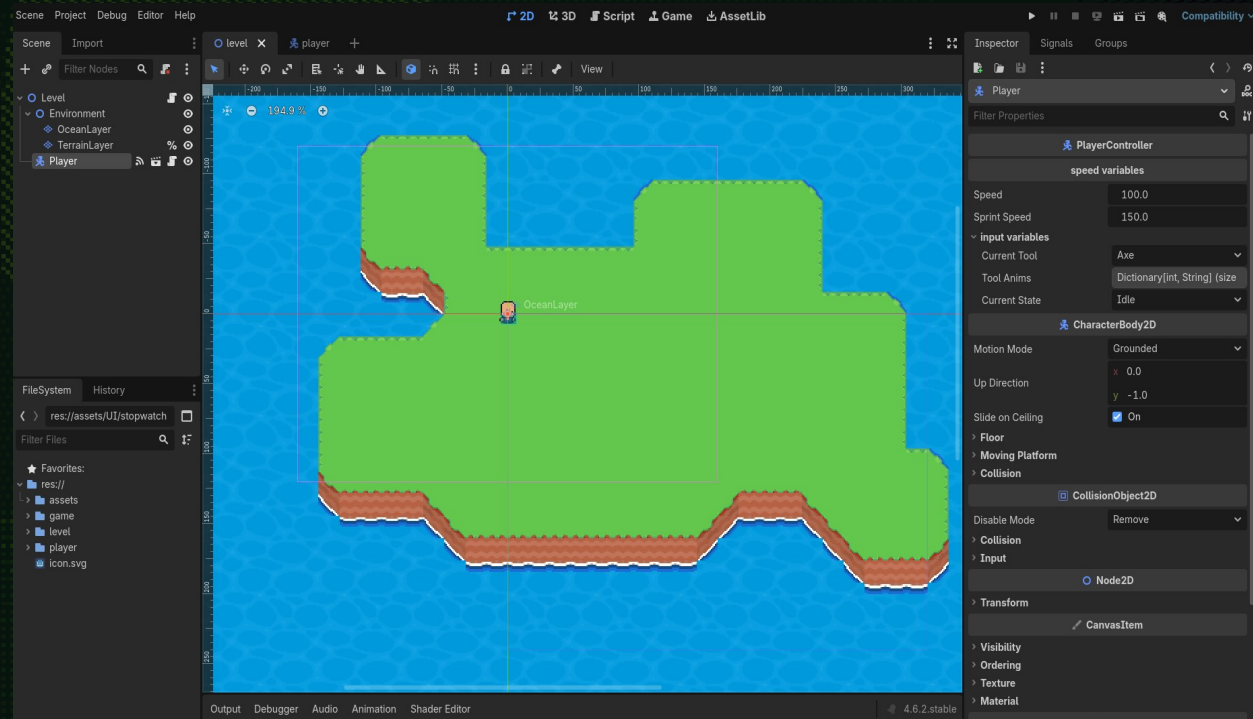
[illegible]

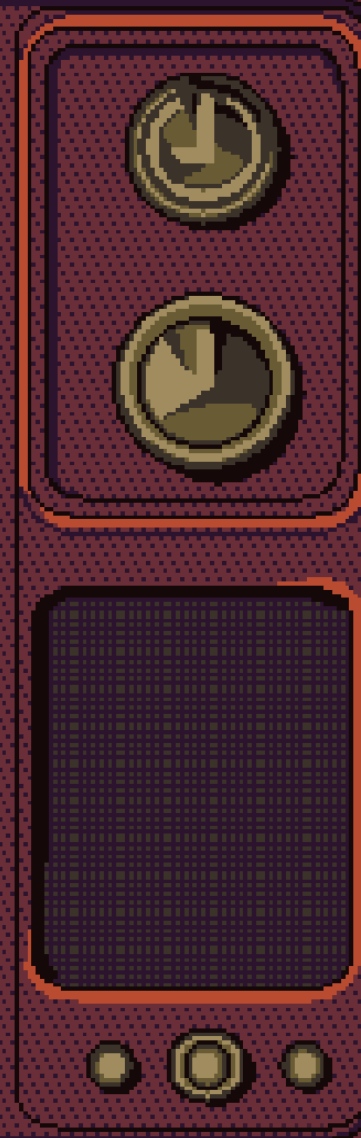
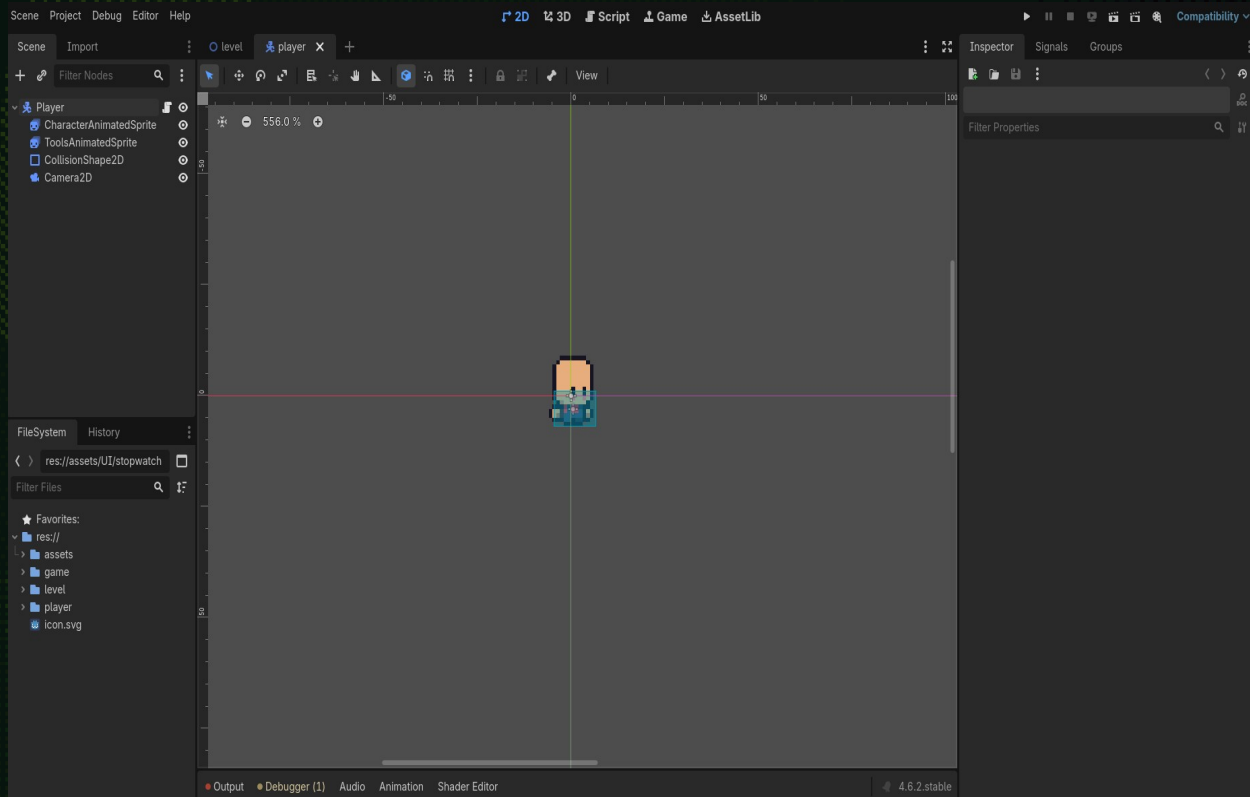
Scene

Jocurile făcute în Godot sunt structurate folosind scene reutilizabile. O scenă poate să fie un caracter, un obiect cu care poți interacționa, un meniu, un nivel întreg, etc.

Scenele pot fi imbricate, astfel putem pune scenele unui caracter, inamici și obiecte într-o scenă nivel.

Exemple de scene





Noduri

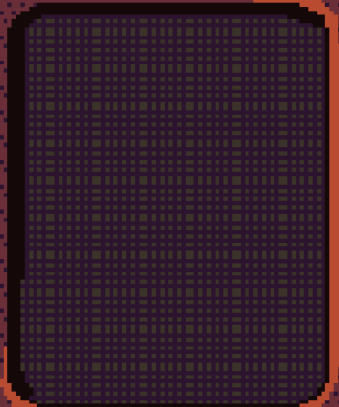


O scenă este compusă din noduri.

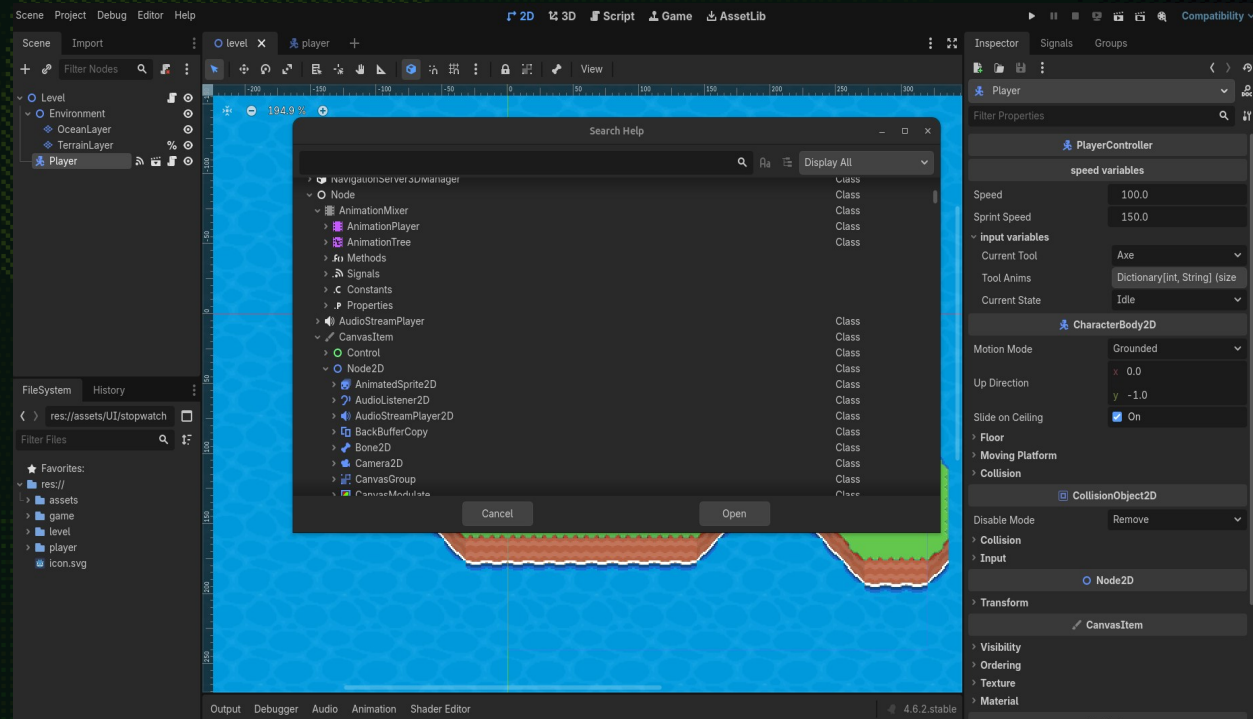
Nodurile în Godot sunt cele mai mici obiecte de sine stătătoare pe care le aranjezi astfel încât să compui o scenă.

Prin atașarea unui script unui nod, putem să extindem funcționalitatea unui nod cu propriul nostru cod.

Putem accesa meniul help folosind tasta F1 sau RCLICK > open documentation pe un nod pentru a afla mai multe informații.



Exemple de noduri



SceneProjectDebugEditorHelp

2D3DScriptGameAssetLib

SceneImport

Filter Nodes

Level

Environment

OceanLayer

TerrainLayer

Player

FileSearchDebug

CharacterBody2D

Open in Online DocsSearch Help

Filter Scripts

level.gd

player_controller.gd

Array

CharacterBody2D

Class: CharacterBody2D

Inherits: PhysicsBody2D < CollisionObject2D < Node2D < CanvasItem < Node < Object

Inherited by: PlayerController

A 2D physics body specialized for characters moved by script.

Description

CharacterBody2D is a specialized class for physics bodies that are meant to be user-controlled. They are not affected by physics at all, but they affect other physics bodies in their path. They are mainly used to provide high-level API to move objects with wall and slope detection (move_and_slide() method) in addition to the general collision detection provided by PhysicsBody2D.move_and_collide(). This makes it useful for highly configurable physics bodies that must move in specific ways and collide with the world, as is often the case with user-controlled characters.

For game objects that don't require complex movement or collision detection, such as moving platforms, AnimatableBody2D is simpler to configure.

Online Tutorials

- Physics introduction
- Troubleshooting physics issues
- Kinematic character (2D)
- Using CharacterBody2D
- 2D_Kinematic_Character_Demo
- 2D_Platformer_Demo

Properties

bool floor_block_on_wall [default: true]

bool floor_constant_speed [default: false]

float floor_max_angle [default: 0.7853982]

InspectorSignalsGroups

Player

Filter Properties

PlayerController

speed variables

Speed100.0

Sprint Speed150.0

input variables

Current ToolAxe

Tool AnimsDictionary[int, String] (size)

Current StateIdle

CharacterBody2D

Motion ModeGrounded

Up Directionx 0.0y -1.0

Slide on CeilingOn

Floor

Moving Platform

Collision

CollisionObject2D

Disable ModeRemove

Collision

Input

Node2D

Transform

CanvasItem

Visibility

Ordering

Texture

Material

FileSystemHistory

res://assets/UI/stopwatch

Filter Files

Favorites:

res//

assets

game

level

player

icon.svg

Top

Description

Properties

Methods

Enumerations

Property Descriptions

Method Descriptions

OutputDebuggerAudioAnimationShader Editor

4.6.2.stable

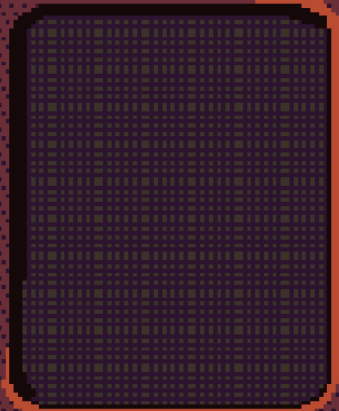


Semnale

Semnalele sunt folosite pentru a facilita comunicarea între noduri. De exemplu când un caracter atinge un obiect putem să transmitem un semnal pentru a crește un scor.

Există de asemenea și semnale built-in pentru noduri pe care le putem vizualiza în engine(de exemplu un buton când este apăsat emite un semnal).

În plus, semnalele sunt folosite pentru a implementa design patterns și programare orientată pe obiecte, despre care vom vorbi la următoarele ședințe.



Exemple de semnale

Class: ➔ Signal

A built-in type representing a signal of an Object.

Description

Signal is a built-in Variant type that represents a signal of an Object instance. Like all Variant types, it can be stored in variables and passed to functions. Signals allow all connected Callable (and by extension their respective objects) to listen and react to events, without directly referencing one another. This keeps the code flexible and easier to manage. You can check whether an Object has a given signal name using `Object.has_signal()`.

In GDScript, signals can be declared with the `signal` keyword. In C#, you may use the `[Signal]` attribute on a delegate.

```
signal attacked
```

```
# Additional arguments may be declared.  
# These arguments must be passed when the signal is emitted.  
signal item_dropped(item_name, amount)
```

Connecting signals is one of the most common operations in Godot and the API gives many options to do so, which are described further down. The code block below shows the recommended approach.

```
func _ready():  
    var button = Button.new()  
    # 'button_down' here is a Signal Variant type. We therefore  
    call the Signal.connect() method, not Object.connect().
```



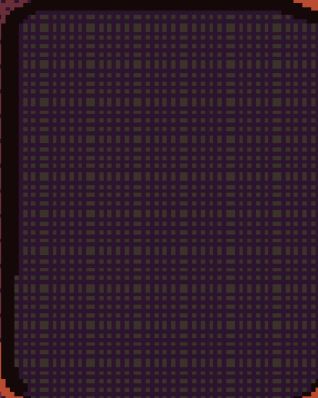
Animation Shader Editor

4.6.2.stable

Connect...

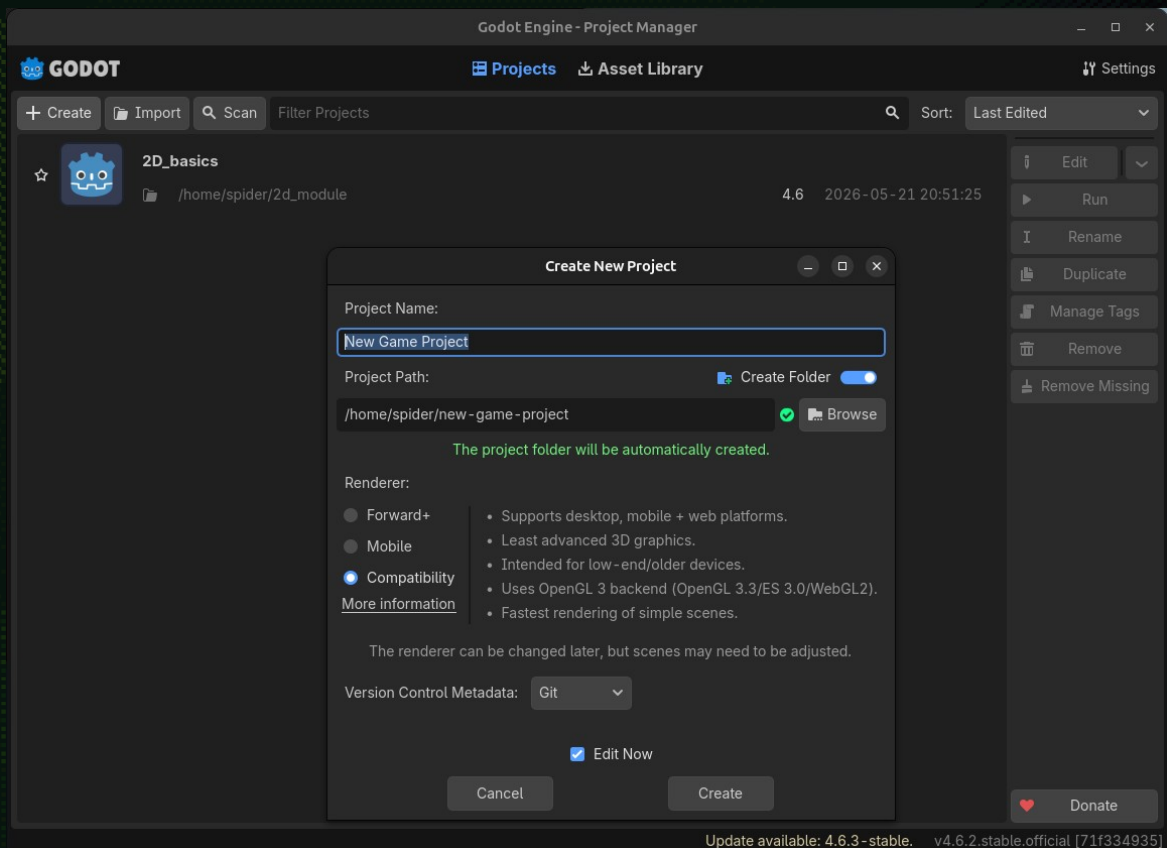
```
PlayerController  
- tool_used(tool: PlayerController.Tool, pos: Vector2)  
CollisionObject2D  
- input_event(viewport: Node, event: InputEvent, shape_idx: int)  
- mouse_entered()  
- mouse_exited()  
- mouse_shape_entered(shape_idx: int)  
- mouse_shape_exited(shape_idx: int)  
CanvasItem  
- draw()  
- hidden()  
- item_rect_changed()  
- visibility_changed()  
Node  
- child_entered_tree(node: Node)  
- child_exiting_tree(node: Node)  
- child_order_changed()  
- editor_description_changed(node: Node)  
- editor_state_changed()  
- ready()  
- renamed()  
- replacing_by(node: Node)  
- tree_entered()  
- tree_exited()  
- tree_exiting()  
Object  
- property_list_changed()  
- script_changed()
```

```
1  extends Node2D
2
3  @onready var player: PlayerController = $"../Player"
4  signal new_environment_created()
5
6
7  # Called when the node enters the scene tree for the first time.
8  func _ready() -> void:
9      # listen for emit() from the player scene and call _on_tool_used() when received
10     player.tool_used.connect(_on_tool_used)
11
12     # broadcast signal to listeners
13     new_environment_created.emit()
14
15  func _on_tool_used() -> void:
16     pass
17
```



Interfața și navigarea

Când deschideți Godot, veți vedea proiect manager-ul unde puteți crea proiecte, instala proiecte demo din asset library și schimba setările de aspect ale engine-ului.

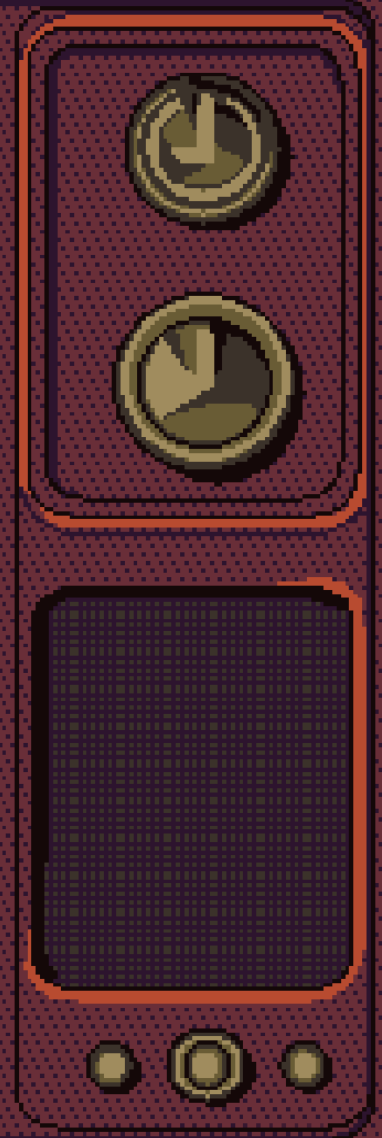


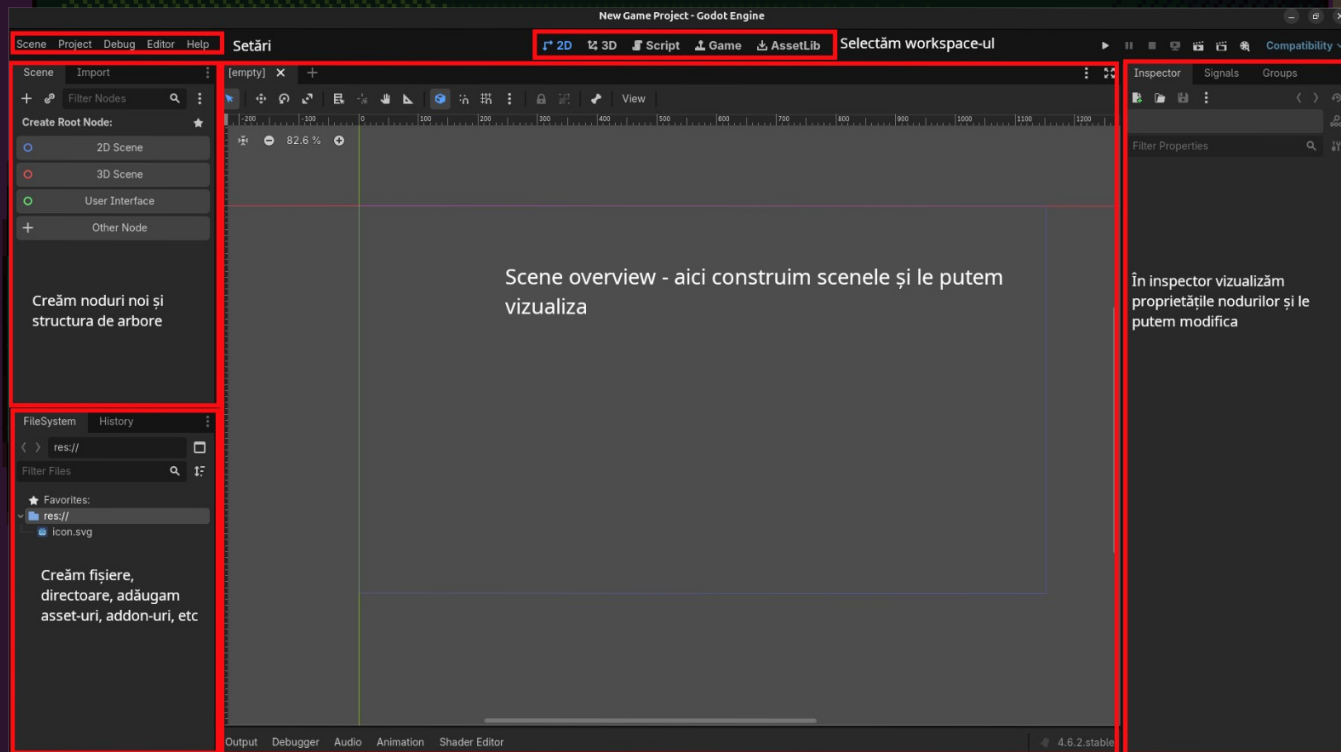
Moduri de rendering

Godot dispune de trei moduri de rendering:

- Forward+ - folosit pentru jocuri cu grafici high-end/ realiste, optimizat pentru scene complexe;
- Mobile - folosit pentru dispozitive mobile și desktop, afișează scene simple mai rapid.
- Compatibility - folosit pentru toate platformele, inclusiv web deoarece este gândit pentru dispozitive low-end.

Vom folosi compatibility pentru a putea exporta jocul și pe web. (în general, dacă este posibil, este bine să faci export pe web deoarece oamenii nu trebuie să downloadeze jocul și o să ai parte de o audiență mai mare)





VCS: lucrul cu `git` și `GitHub`

VCS(version control system) este software care ține un istoric al schimbărilor dintr-un proiect. Cel mai folosit VCS este git.

Avantajul folosirii unui VCS este că simplifică lucrul în echipă și creează backup-uri ale proiectului. Platforma cea mai folosită pentru a hosta un repository(un proiect software) este GitHub.

Requirements: git instalat pe sistem, cont de GitHub

GitHub Desktop

GitHub Desktop este un software care simplifică
lucrul cu git pentru utilizatorii Windows/MacOS.

Tutorial GitHub

Top repositories

New

Find a repository...

- 0xspiderr/gbjam13_gamejam
- 0xspiderr/awawa-game-steam
- 0xspiderr/multiplayer-fps
- 0xspiderr/itec-2026
- 0xspiderr/godot-visual-prog-fps
- 0xspiderr/enet-chess
- 0xspiderr/esp32-cam-prj

Show more

Pe butonul
'New' creăm un
repository nou

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 Oxspiderr ▾

Repository name *

Great repository names are short and memorable. How about [urban-goggles?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

Adăugăm setări
Pentru repository

test_tutorial Private

Watch 0 Fork 0 Star 0



Start coding with Codespaces

Add a README file and start coding in a secure, configurable, and dedicated development environment.

Create a codespace



Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

HTTPS SSH

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# test_tutorial" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/oxspiderr/test_tutorial.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/oxspiderr/test_tutorial.git
git branch -M main
git push -u origin main
```

ProTip! Use the URL for this page when adding GitHub as a remote.

Setup
repository(cu
GitHub
desktop tot
procesul
acesta este
automatizat)

Comenzi git

În folder-ul proiectului nostru trebuie să creem un repository git local. Pentru asta trebuie să deschidem command line-ul(pe Windows git bash).



FileSystem

History

< > res://

Filter Files

★ Favorites:

▼ res://

🖼 icon.svg

+ Create New

📁 Expand Folder

▼ Expand Hierarchy

▶ Collapse Hierarchy

📄 Copy Path

Ctrl+Shift+C

Copy Absolute Path

Ctrl+Alt+C

★ Add to Favorites

➤ Open in Terminal

Ctrl+Alt+T

📂 Open in File Manager

Ctrl+Alt+R

Output

Debugger

Aut



spider@spider: ~/github-tutorial/



```
spider@spider:~/github-tutorial/$ git init
```

hint: Using 'master' as the name for the initial branch. This default branch name

hint: is subject to change. To configure the initial branch name to use in all

hint: of your new repositories, which will suppress this warning, call:

hint:

hint: git config --global init.defaultBranch <name>

hint:

hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and

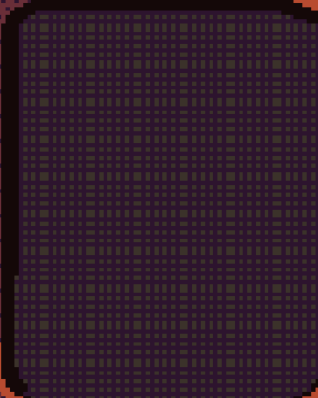
hint: 'development'. The just-created branch can be renamed via this command:

hint:

hint: git branch -m <name>

Initialized empty Git repository in /home/spider/github-tutorial/.git/

```
spider@spider:~/github-tutorial/$ git remote add origin https://github.com/0xspiderr/test_tutorial
```



Câteva comenzi importante de git

`git init` - crează un repository local

`git remote add origin <url>` - conectează repository-ul nostru local cu locul unde este hostat (GitHub în cazul nostru).

`origin` - numele default al repository-ului remote



```
spider@spider:~/github-tutorial/$ git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
.editorconfig
```

```
.gitattributes
```

```
.gitignore
```

```
icon.svg
```

```
icon.svg.import
```

```
project.godot
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

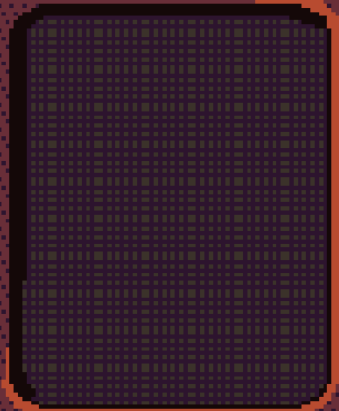
```
spider@spider:~/github-tutorial/$ git add .
```

`git status` - afișează branch-ul curent pe care ne aflăm.

Un branch pe git este un workspace separat pe care putem lucra la alte feature-uri și este foarte util pentru lucrul în echipă. În general workflow-ul pe care l-am folosit este să avem un branch separat fiecare membru din echipă și să îl sincronizăm cu branch-ul principal(main).

`git add <nume_fișier>` - adaugă fișiere modificate pe care urmează să le commit-uim

`git add .` - pentru a adăuga toate fișierele



```
spider@spider:~/github-tutorial/$ git commit -m "first commit"
```

```
[main (root-commit) 1a57092] first commit
```

```
6 files changed, 78 insertions(+)
```

```
create mode 100644 .editorconfig
```

```
create mode 100644 .gitattributes
```

```
create mode 100644 .gitignore
```

```
create mode 100644 icon.svg
```

```
create mode 100644 icon.svg.import
```

```
create mode 100644 project.godot
```

```
spider@spider:~/github-tutorial/$ git push -u origin main
```

```
Enumerating objects: 8, done.
```

```
Counting objects: 100% (8/8), done.
```

```
Delta compression using up to 16 threads
```

```
Compressing objects: 100% (6/6), done.
```

```
Writing objects: 100% (8/8), 1.86 KiB | 1.86 MiB/s, done.
```

```
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
```

```
To https://github.com/0xspiderr/test_tutorial
```

```
* [new branch]      main -> main
```

```
branch 'main' set up to track 'origin/main'.
```

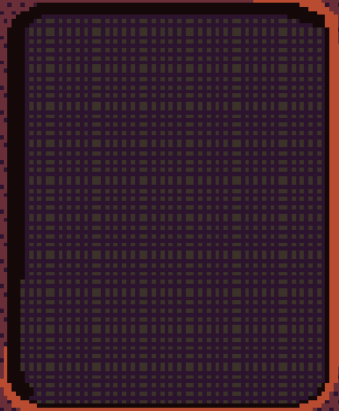
```
spider@spider:~/github-tutorial/$
```

`git commit -m <mesaj de commit>` - crează un "snapshot" al proiectului și adaugă un mesaj. În general ar trebui să punem mesaje bazat pe feature-urile adăugate sau acțiunile făcute(delete/update, etc).

`git push -u origin main` - publică schimbările(commit-urile) pe remote(GitHub).

Argumentele `-u origin main` sunt necesare doar la primul commit, la următorul commit putem scrie direct:

`git push`

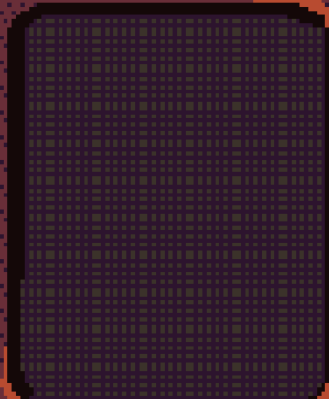


Alte comenzi utile

`git pull` - preia toate schimbările de pe un repository remote. Util când lucrezi cu mai mulți oameni sau ai mai multe device-uri de pe care lucrezi și vrei să le sincronizezi. De asemenea folosit pentru sincronizare de pe alte branch-uri.

`git help <nume_comandă>` - afișează text ajutător despre comanda dată ca argument

`git clone <url>` - downloadează local un repository remote(puteți să dați clone la repository-ul cu prezentări)



Workflow git

1. Pașii de inițializare(ii facem o singură dată):

```
git init
```

```
git remote add origin <url>
```

```
git add .
```

```
Git commit -m <mesaj>
```

```
git push -u origin main
```

2. Pașii în development

```
git pull
```

```
git add .
```

```
git commit -m <mesaj>
```

```
git push
```

Dacă lucrați solo, o să vă descurcați 99% din timp doar cu aceste 4 comenzi. Este esențial să dăm git pull de fiecare dată când scriem o comandă dacă mai avem alte device-uri de pe care lucrăm/ lucrăm cu alți oameni pe același branch, dacă nu o să avem merge conflicts.